Student Organization Management System

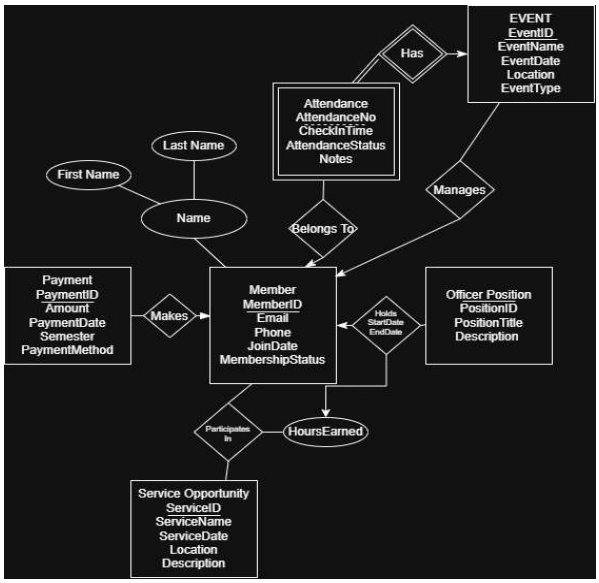
Project Milestone 2

Bailey French (Smith)

CSC 315

1. Revised E-R Diagram

The E-R diagram from Project Milestone 1 was reviewed and revised before converting the design into relational schemas. The revised model represents members, events, attendance, payments, service opportunities, and officer positions, along with the relationships between them.



2. Relational Schemas

The E-R model was converted into relational schemas for implementation in MySQL. Each main entity became a relation, while relationships that needed their own data were converted into separate relations. Primary keys are used to uniquely identify records, and foreign keys are used to connect related records between tables.

Member(

member_id PK,

first_name,

last_name,

email UNIQUE,

phone,

join_date,

membership_status

)

Event(

event_id PK,

event_name,

event_type,

event_date,

event_time,

location,

description,

manager_id FK → Member.member_id

)

Attendance(

member_id PK, FK → Member.member_id,

event_id PK, FK → Event.event_id,

attendance_status,

check_in_time,

notes

)

Payment(

payment_id PK,
member_id FK → Member.member_id,
amount,
payment_date,
semester,
payment_method,
payment_type,
payment_status
)

ServiceOpportunity(
service_id PK,
service_name,
service_date,
location,
description
)

MemberService(
member_id PK, FK → Member.member_id,
service_id PK, FK → ServiceOpportunity.service_id,
hours_completed
)

OfficerPosition(
position_id PK,
position_title UNIQUE,

description

)

MemberOfficerPosition(

member_id PK, FK → Member.member_id,

position_id PK, FK → OfficerPosition.position_id,

start_date PK,

end_date

)

Attendance uses a composite primary key consisting of `member_id` and `event_id`. MemberService uses `member_id` and `service_id` as its composite primary key. MemberOfficerPosition uses `member_id`, `position_id`, and `start_date` as its composite primary key. These composite keys allow the database to represent relationships between records without unnecessarily duplicating information.

3. Normalization

The database was normalized to reduce duplicate data and help keep the information consistent. The relations satisfy First Normal Form (1NF) because each field stores a single value and there are no repeating groups. The relations satisfy Second Normal Form (2NF) because the non-key attributes depend on the entire primary key. This is especially important in tables that use composite primary keys, such as Attendance and MemberService. The database also satisfies Third Normal Form (3NF) because the non-key attributes depend on the primary key instead of depending on other non-key attributes. Information about members, events, payments, service opportunities, and officer positions is kept in separate tables instead of repeating the same information in multiple places. Many-to-many relationships from the E-R model were also separated into their own relations. The relationship between Member and ServiceOpportunity became MemberService, which stores the hours completed by a member for a particular service opportunity. The relationship between Member and OfficerPosition became MemberOfficerPosition, which stores the dates that a member holds a specific officer position. Attendance connects Member and Event and stores information about a member's attendance at an event. The Manages relationship between Member and Event was implemented using `manager_id` as a foreign key in Event. This allows one member to manage multiple events while each event can be connected to its primary manager.

4. SQL Implementation

The Student Organization Management System was implemented using MySQL and MySQL Workbench. The database was created as `student_org_management`. SQL scripts were separated into files for creating the database structure, inserting sample data, implementing the functional requirements, and testing database constraints.

The main SQL files used in the project are:

schema.sql - Creates the tables, keys, constraints, and views

data.sql - Inserts sample data into the database

queries.sql - Contains SQL for the functional requirements

tests.sql - Tests database constraints and integrity

The final database contains eight tables:

Member

Event

Attendance

Payment

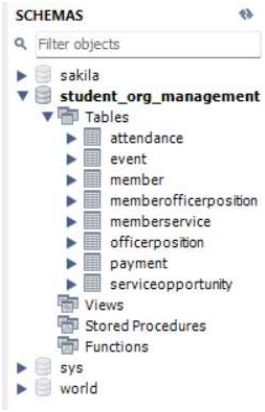
ServiceOpportunity

MemberService

OfficerPosition

MemberOfficerPosition

Primary keys were created to uniquely identify records. Foreign keys were used to maintain relationships between the tables. Other constraints were also used where needed, including UNIQUE and CHECK constraints. For example, member email addresses are unique and payment amounts cannot be negative.



5. Functional Requirements

The functional requirements from Project Milestone 1 were implemented using SQL statements and queries. The following explains how each requirement is handled by the database.

FR1 – Add a new member

An officer can add a new member by inserting a record into the Member table. The member's name, email, phone number, join date, and membership status can be recorded.

FR2 – Update member information

An officer can update an existing member using an UPDATE statement. This can be used to change information such as the member's phone number or membership status.

FR3 – Create and manage an event

An event can be inserted into the Event table with information including the event name, type, date, time, location, and description. The `manager_id` foreign key connects the event to the member responsible for managing it.

FR4 – Record event attendance

Attendance is recorded in the Attendance table. Each record connects a member to an event and can include attendance status, check-in time, and notes.

FR5 – Record a dues payment

Payments are stored in the Payment table. The database records the member, amount, payment date, semester, payment method, payment type, and payment status.

FR6 – View members who have or have not paid dues

A query using Member and Payment can display whether each member has paid dues for a selected semester. A LEFT JOIN allows members without a matching payment to still appear in the results.

FR7 – Create a service opportunity and record participants

Service opportunities are stored in ServiceOpportunity. Members participating in an opportunity are connected to it through MemberService.

FR8 – Record service hours

The `hours_completed` attribute in MemberService records the number of service hours earned by a member for a particular service opportunity.

FR9 – View total service hours

An aggregate query using `SUM()` calculates the total service hours earned by each member.

FR10 – Assign an officer position

Members are assigned to officer positions through MemberOfficerPosition. This relation also records the beginning and ending dates of the member's term.

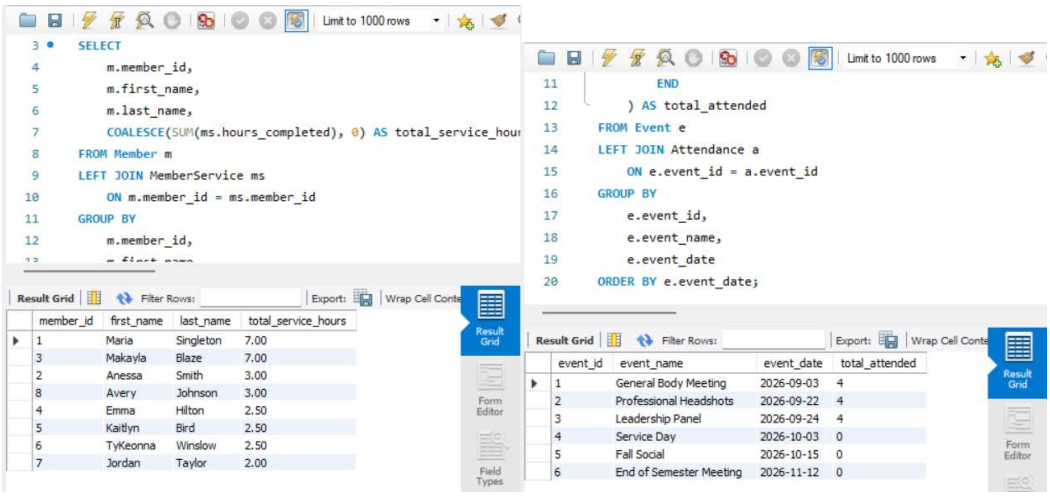
FR11 – Display event attendance totals

An aggregate query using `COUNT()` displays the total number of members who attended each event.

FR12 – Display dues totals

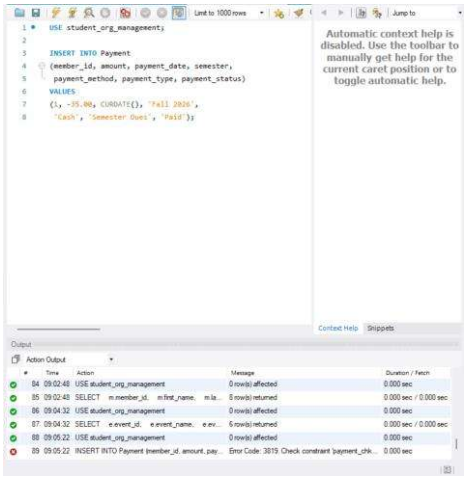
An aggregate query using `SUM()` calculates the total amount of dues recorded for a selected semester.

These queries demonstrate inserts, updates, joins, and aggregate operations while also showing how the database supports the original requirements from Project Milestone 1.



6. Testing and Constraints

The database was tested to make sure valid data could be entered while invalid data was rejected. Positive and negative tests were performed using the SQL statements in `tests.sql`. A positive test was performed by inserting a valid record and confirming that the record could be selected from the database. The test record could then be deleted after the test was completed. Negative tests were also performed to verify database constraints. A duplicate email address was tested against the UNIQUE constraint on Member. MySQL rejected the duplicate value, showing that two members cannot accidentally use the same email address. A foreign key test was performed by attempting to create a record connected to a member that did not exist. MySQL rejected the record because the referenced member could not be found. The CHECK constraint on Payment was also tested by attempting to insert a negative payment amount. MySQL rejected the invalid amount because the Payment table requires the amount to be greater than or equal to zero. These tests demonstrate that the database is able to maintain entity integrity, referential integrity, and basic domain constraints.



7. GitHub Repository

The project files are stored in a GitHub repository so the SQL implementation and project documentation can be maintained together. The repository contains the SQL scripts used to create, populate, query, and test the database, along with the project documentation. Meaningful commits are used to show the development of the project instead of uploading the entire completed project in one commit. The README provides a short description of the Student Organization Management System and explains the purpose of the major project files.

baileynichole819/CSC315-Student-Organization-Database: Student Organization Management System database project for CSC 315